Apogee: Planar Two-Stage Ascent to Circular LEO

Mathematical Formulation, Numerical Methods, and Implementation Report

Apogee Development Team

January 12, 2026

Abstract

This report details the mathematical models and numerical algorithms governing the **Apogee** space mission simulator. The system models the ascent of a two-stage liquid-propellant rocket into a circular Low Earth Orbit (LEO) considering variable mass, non-constant gravity, and atmospheric drag based on the US Standard Atmosphere 1976 (USSA76). The flight is modeled as a hybrid dynamical system with discrete event-driven phase transitions. The trajectory optimization problem is solved using a custom implementation of a Levenberg-Marquardt algorithm with Broyden rank-1 updates. This document serves as the authoritative reference for the codebase; all equations are numbered and explicitly mapped to their software implementations.

Contents

1 Project Structure and Architecture **3**

2 Coordinate System and State Vector **3**

3 Equations of Motion (EOM) **3**
- 3.1 Kinematics . . . 3
- 3.2 Forces . . . 4
- 3.3 Dynamics Derivation . . . 4
- 3.4 Mass Flow . . . 4
- 3.5 Numerical Regularization . . . 5

4 Atmosphere and Aerodynamics **5**
- 4.1 US Standard Atmosphere 1976 . . . 5
- 4.2 Aerodynamic Drag . . . 5

5 Hybrid System Simulation **5**
- 5.1 Phase A: Vertical Ascent . . . 5
- 5.2 Phase B: Gravity Turn (Stage 1) . . . 6
- 5.3 Phase C: Coast . . . 6
- 5.4 Phase D: Stage 2 Burn . . . 6

6 Numerical Methods **6**
- 6.1 Integration (Tsit5) . . . 6
- 6.2 Event Handling (Newton-Raphson) . . . 6

7 Orbital Mechanics **6**

8 Optimization Algorithm (Shooting Method) 7
8.1 Problem Formulation 7
8.2 Algorithm: Levenberg-Marquardt with Broyden 7

1 Project Structure and Architecture

The `Apogee` project is organized as a monorepo using `uv` workspaces to ensure modularity and clean separation of concerns.

`packages/apogee-physics` The core numerical engine.

- `dynamics.py`: Contains the Equations of Motion (EOM) derived in Section 3.
- `atmosphere.py`: Implements the USSA76 atmospheric model (Section 4).
- `simulate.py`: Manages the hybrid system integration and event handling (Section 5).
- `shooting.py`: Implements the optimization algorithms (Section 8).
- `orbit.py`: Calculates Keplerian orbital elements (Section 7).

`packages/apogee-launch` The simulation runner and CLI tools.

`packages/apogee-api` A FastAPI backend for serving simulation results.

`packages/apogee-orbit` Future module for long-term orbit propagation and perturbations.

2 Coordinate System and State Vector

We define the motion in a 2D inertial plane centered at the Earth. We assume a non-rotating Earth for the ascent phase purely to simplify the equations to a planar model, although the physics engine is extensible.

The state vector $\boldsymbol{y}(t)$ consists of 5 variables:

$$\boldsymbol{y}(t) = \begin{bmatrix} r(t) \\ \lambda(t) \\ v(t) \\ \gamma(t) \\ m(t) \end{bmatrix} \tag{1}$$

Where:

- $r(t)$: Geocentric radius (distance from Earth center) [m].
- $\lambda(t)$: Downrange central angle [rad].
- $v(t)$: Velocity magnitude (speed) [m/s].
- $\gamma(t)$: Flight-path angle (FPA), measured from the local horizontal [rad]. $\gamma > 0$ implies ascent.
- $m(t)$: Vehicle total mass [kg].

3 Equations of Motion (EOM)

3.1 Kinematics

From the definition of polar coordinates, the velocity vector $\boldsymbol{v}$ can be decomposed into radial ($\hat{e}_r$) and tangential ($\hat{e}_\lambda$) components:

$$\boldsymbol{v} = v \sin\gamma \, \hat{e}_r + v \cos\gamma \, \hat{e}_\lambda \tag{2}$$

This yields the kinematic differential equations:

$$\frac{dr}{dt} = v \sin \gamma \tag{3}$$

$$\frac{d\lambda}{dt} = \frac{v \cos \gamma}{r} \tag{4}$$

3.2 Forces

The vehicle is subject to three primary forces:

1. **Thrust (T)**: Acts along the vehicle's longitudinal axis. The steering angle α is defined as the angle between the thrust vector and the velocity vector.
2. **Drag (D)**: Acts opposite to the velocity vector.
3. **Gravity (g)**: Acts towards the center of the Earth.

3.3 Dynamics Derivation

Newton's Second Law in the path coordinate system (tangent $\hat{t}$ and normal $\hat{n}$ to the trajectory) states:

$$\sum F_t = m \frac{dv}{dt} \tag{5}$$

$$\sum F_n = mv \frac{d\gamma}{dt} - m \frac{v^2}{r} \cos \gamma \quad \text{(includes centrifugal term)} \tag{6}$$

Resolving forces:

- **Tangential**: Thrust component $T \cos \alpha$, Drag $-D$, Gravity component $-mg \sin \gamma$.
- **Normal**: Thrust component $T \sin \alpha$, Gravity component $-mg \cos \gamma$.

Substituting $g = \mu / r^2$, we obtain the dynamic equations:
Speed Equation:

$$\frac{dv}{dt} = \frac{T}{m} \cos \alpha - \frac{D}{m} - \frac{\mu}{r^2} \sin \gamma \tag{7}$$

Flight-Path Angle Equation:

$$\frac{d\gamma}{dt} = \frac{T}{mv} \sin \alpha + \left(\frac{v}{r} - \frac{\mu}{r^2 v} \right) \cos \gamma \tag{8}$$

3.4 Mass Flow

The mass depletion is governed by the rocket equation:

$$\frac{dm}{dt} = -\frac{T}{I_{sp} g_0} \tag{9}$$

where I_{sp} is the specific impulse [s] and g_0 is standard gravity (9.80665 m/s^2).

3.5 Numerical Regularization

Equations (8) contains terms with $1/v$. At launch ($t = 0, v = 0$), this causes a singularity. In the code implementation (`dynamics.py`), we apply a regularization technique.

We introduce a small epsilon velocity v_ϵ. We define a regularized inverse velocity:

$$\frac{1}{v}_{\text{reg}} = \frac{v}{v^2 + v_\epsilon^2} \tag{10}$$

This term behaves like $1/v$ for large v, but goes to 0 as $v \to 0$, preventing numerical explosion.

We also guard against $r \to 0$ (though physically impossible in flight) using:

$$r_{\text{safe}} = \max(r, 0.99R_E) \tag{11}$$

The implemented regularized $\dot{\gamma}$ equation is:

$$\frac{d\gamma}{dt} \approx \frac{T \sin\alpha}{m}\left(\frac{1}{v}_{\text{reg}}\right) + \left(\frac{v}{r_{\text{safe}}} - \frac{\mu}{r_{\text{safe}}^2}\left(\frac{1}{v}_{\text{reg}}\right)\right)\cos\gamma \tag{12}$$

4 Atmosphere and Aerodynamics

4.1 US Standard Atmosphere 1976

We compute air density $\rho(h)$ and speed of sound $a_s(h)$ using the USSA76 model. The implementation builds a lookup table and uses linear interpolation.

$$\rho(h) = \text{Interp}(h, \text{USSA76}_\rho) \tag{13}$$

4.2 Aerodynamic Drag

The Mach number is $M = v/a_s(h)$. The drag force is:

$$D = \frac{1}{2}\rho v^2 C_D(M) A_{\text{ref}} \tag{14}$$

where $C_D(M)$ is the drag coefficient (potentially varying with Mach number) and A_{ref} is the vehicle reference area.

5 Hybrid System Simulation

The flight is modeled as a sequence of phases, each governed by specific constraints or parameters.

5.1 Phase A: Vertical Ascent

- **Constraint**: $\gamma = \pi/2$ (constant).
- **Dynamics**: $\dot{\gamma} = 0$. $\dot{\lambda} = 0$.
- **Termination Event**: Altitude reaches $h_{\text{pitchover}}$.
- **Transition**: Instantaneous change of γ to $\pi/2 - \theta_0$.

5.2 Phase B: Gravity Turn (Stage 1)

- **Control**: $\alpha = 0$ (Thrust aligned with velocity).
- **Dynamics**: Full EOM (3-9).
- **Termination Event**: Mass reaches $m_{1,\text{end}}$ (Stage 1 burnout).
- **Transition**: Stage separation ($m \to m - m_{1,\text{dry}}$).

5.3 Phase C: Coast

- **Control**: $T = 0$, $\dot{m} = 0$.
- **Dynamics**: Ballistic motion (Drag + Gravity).
- **Termination Event**: Time duration t_{coast} elapses.

5.4 Phase D: Stage 2 Burn

- **Control**: $\alpha = \alpha_2$ (Constant steering angle).
- **Termination Event**: Time duration t_{burn2} or Fuel Depletion.

6 Numerical Methods

6.1 Integration (Tsit5)

We use the **Tsitouras 5(4)** method, an explicit Runge-Kutta method. It is an adaptive step-size solver.

- **How it works**: It computes the solution at $t + \Delta t$ using a 5th-order formula and a 4th-order formula. The difference between these two estimates provides an error estimate.
- **Adaptivity**: If the error is greater than the tolerance (`rtol`, `atol`), the step size Δt is reduced. If the error is very small, Δt is increased to speed up computation.

6.2 Event Handling (Newton-Raphson)

To detect events (like reaching a specific altitude), we solve $g(\boldsymbol{y}, t) = 0$. The solver steps over the event (where g changes sign) and then uses the **Newton-Raphson** method to iteratively find the exact time t^* where $g(t^*) = 0$.

7 Orbital Mechanics

At injection, we calculate the osculating orbital elements:

Specific Mechanical Energy:

$$\varepsilon = \frac{v^2}{2} - \frac{\mu}{r} \tag{15}$$

Specific Angular Momentum:

$$h_{\text{ang}} = rv\cos\gamma \tag{16}$$

Eccentricity:

$$e = \sqrt{1 + \frac{2\varepsilon h_{\text{ang}}^2}{\mu^2}} \tag{17}$$

Semi-major Axis:

$$a = -\frac{\mu}{2\varepsilon} \tag{18}$$

8 Optimization Algorithm (Shooting Method)

We wish to find the control parameters $\boldsymbol{u}$ that result in a circular orbit at a specific target altitude.

8.1 Problem Formulation

Decision Variables ($\boldsymbol{u}$):

- θ_0: Initial pitch-over angle.
- t_{coast}: Duration of coast phase.
- t_{burn2}: Duration of second burn.
- α_2: Steering angle for second stage.

Residuals ($\boldsymbol{F}(\boldsymbol{u})$): We want the final state to match:

$$\boldsymbol{F}(\boldsymbol{u}) = \begin{bmatrix} (a - r_{\text{target}})/r_{\text{target}} \\ e \\ \gamma \end{bmatrix} \approx \mathbf{0} \tag{19}$$

This enforces correct altitude ($a = r$), circularity ($e = 0$), and zero vertical velocity ($\gamma = 0$).

8.2 Algorithm: Levenberg-Marquardt with Broyden

This is a root-finding algorithm designed to be robust against poor initial guesses.

1. **Variable Transformation**: We map the bounded physical variables $\boldsymbol{u}$ to unbounded variables $\boldsymbol{x}$ using a sigmoid function (Eq. 20). This allows the optimizer to work in $\mathbb{R}^n$ without violating physical constraints.

$$u_i = u_{\min} + (u_{\max} - u_{\min}) \cdot \sigma(x_i) \tag{20}$$

2. **Jacobian Calculation**: We compute the sensitivity matrix (Jacobian) $J = \partial \boldsymbol{F}/\partial \boldsymbol{x}$ using Finite Differences. This tells us how the errors change when we tweak the controls.

3. **Damped Step (Levenberg-Marquardt)**: instead of a pure Newton step ($\Delta \boldsymbol{x} = -J^{-1}\boldsymbol{F}$), we solve:

$$(J^T J + \lambda I)\Delta \boldsymbol{x} = -J^T \boldsymbol{F} \tag{21}$$

   λ is a damping parameter.

   - Large λ: Behaves like Gradient Descent (slow but reliable).
   - Small λ: Behaves like Gauss-Newton (fast convergence near solution).

4. **Broyden Update**: Re-computing the Jacobian with finite differences is expensive (requires 4+ simulations). We use the Broyden rank-1 update to approximate the new Jacobian based on the change in residuals, saving computation time.